

kaggar11/ systolic_4x4arr



A 4x4 Weight Stationary Systolic Array
Implementation



1

Contributor



0

Issues



15

Stars



1

Fork



kaggar11/systolic_4x4arr: A 4x4 Weight Stationary Systolic Array
Implementation

A 4x4 Weight Stationary Systolic Array Implementation - kaggar11/systolic_4x4arr

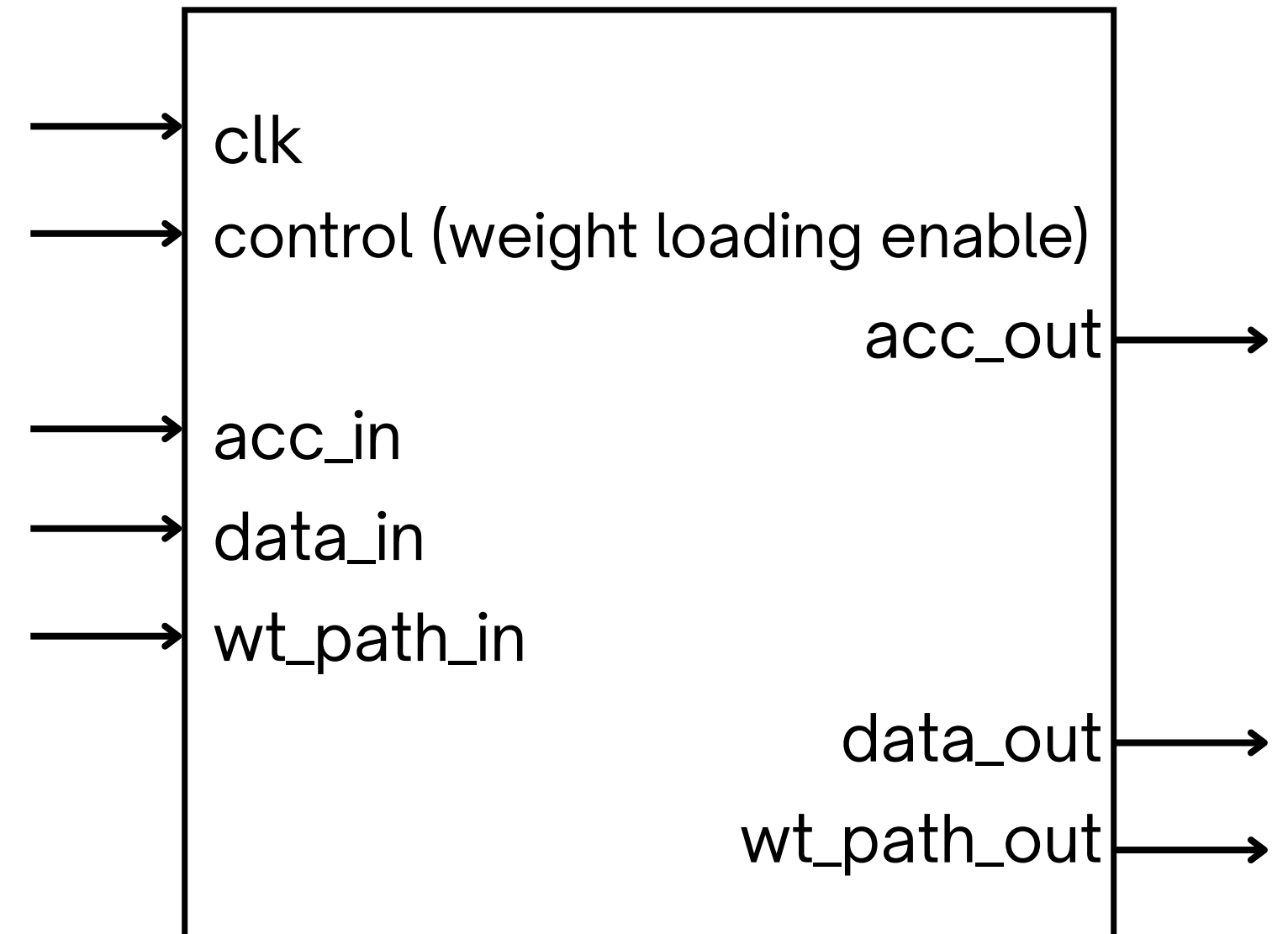


GitHub

Weight-Stationary Systolic Array Implementation in Verilog

What the PE looks like in code

```
module MAC #(parameter bit_width=8, acc_width=32)(  
    clk,  
    control,  
    acc_in,    //a  
    acc_out,   //a+b*c  
    data_in,   //b  
    wt_path_in,    //c  
    data_out,  
    wt_path_out  
);
```



What the PE looks like in code

Variable Declarations

```
input clk;
input control; // control signal used to indicate if it is weight loading or not

input [acc_width-1:0] acc_in; // accumulation in
input [bit_width-1:0] data_in; // data input or activation in
input [bit_width-1:0] wt_path_in; // weight data in
output reg [acc_width-1:0] acc_out; // accumulation out
output reg [bit_width-1:0] data_out; // activation out
output reg [bit_width-1:0] wt_path_out; // weight data out

// A register to store the stationary weights
reg [bit_width-1:0] mac_weight;
wire [acc_width-1:0] mlt;

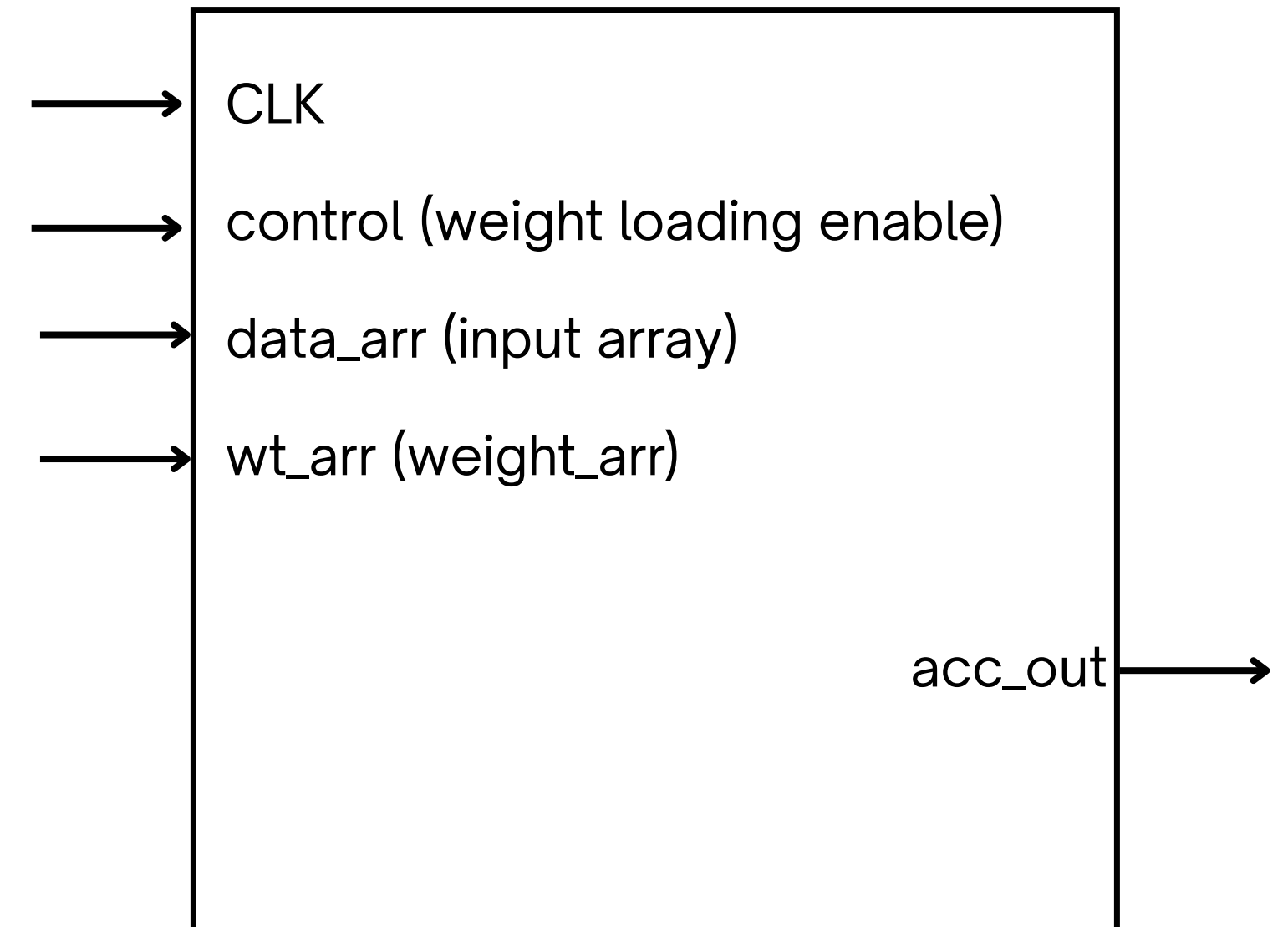
assign mlt = control ? 'h0 : data_in*mac_weight;
```

always@(posedge clk) block

```
// implement your MAC Unit below
always@(posedge clk) begin
    if (control) begin
        data_out    <= 'h0;
        acc_out     <= 'h0;
        mac_weight  <= wt_path_in;
        wt_path_out <= wt_path_in;
    end else begin
        data_out    <= data_in;
        acc_out     <= acc_in + mlt;
    end
end
```

What the MMU looks like in code

```
module MMU#(parameter depth=4, bit_width=8, acc_width=32, size=4)
(
    clk,
    control,
    data_arr,
    wt_arr,
    acc_out
);
    input clk;
    input control;
    input [(bit_width*depth)-1:0] data_arr;
    input [(bit_width*depth)-1:0] wt_arr;
    output reg [acc_width*size-1:0] acc_out;
```

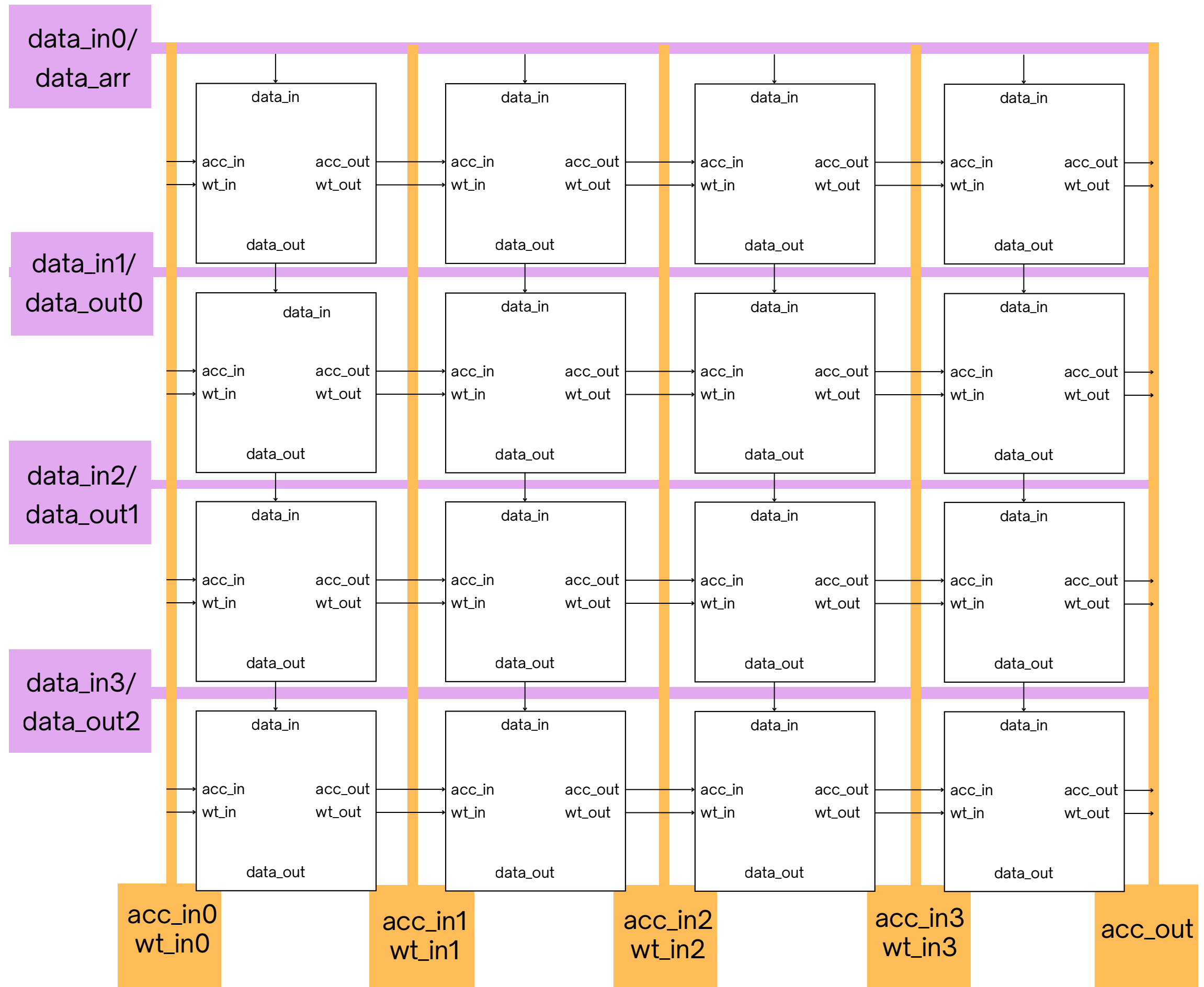


How data is mapped: the whole MMU

```
assign mac_acc_in0 = 'h0;
assign mac_acc_in1 = mac_acc_out0;
assign mac_acc_in2 = mac_acc_out1;
assign mac_acc_in3 = mac_acc_out2;
```

```
assign mac_data_in0 = control ? 'h0 : data_arr;
assign mac_data_in1 = mac_data_out0;
assign mac_data_in2 = mac_data_out1;
assign mac_data_in3 = mac_data_out2;
```

```
// need to shift weights only when control bit i
assign mac_wt_in0 = control ? wt_arr : 'h0;
assign mac_wt_in1 = control ? mac_wt_out0 : 'h0;
assign mac_wt_in2 = control ? mac_wt_out1 : 'h0;
assign mac_wt_in3 = control ? mac_wt_out2 : 'h0;
```



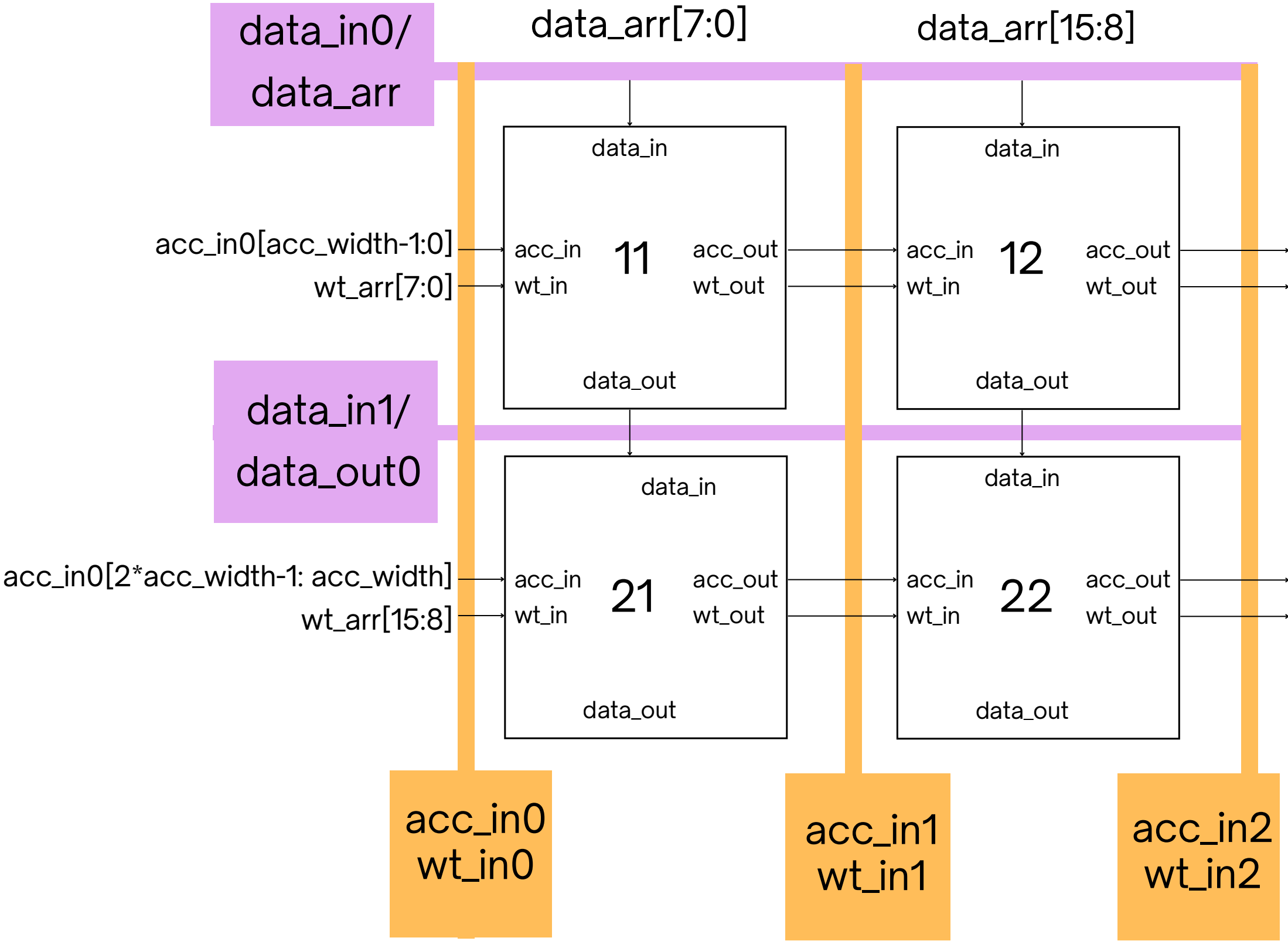
How data is mapped: per-PE

```
MAC u_MAC11 (  
  .clk      (clk),  
  .control  (control),  
  .data_in  (mac_data_in0[7:0]),  
  .wt_path_in (mac_wt_in0[7:0]),  
  .acc_in   (mac_acc_in0[acc_width-1:0]),  
  .data_out  (mac_data_out0[7:0]),  
  .wt_path_out (mac_wt_out0[7:0]),  
  .acc_out  (mac_acc_out0[acc_width-1:0])  
);
```

```
MAC u_MAC12 (  
  .clk      (clk),  
  .control  (control),  
  .data_in  (mac_data_in0[15:8]),  
  .wt_path_in (mac_wt_in1[7:0]),  
  .acc_in   (mac_acc_in1[acc_width-1:0]),  
  .data_out  (mac_data_out0[15:8]),  
  .wt_path_out (mac_wt_out1[7:0]),  
  .acc_out  (mac_acc_out1[acc_width-1:0])  
);
```

```
MAC u_MAC21 (  
  .clk      (clk),  
  .control  (control),  
  .data_in  (mac_data_in1[7:0]),  
  .wt_path_in (mac_wt_in0[15:8]),  
  .acc_in   (mac_acc_in0[2*acc_width-1:acc_width]),  
  .data_out  (mac_data_out1[7:0]),  
  .wt_path_out (mac_wt_out0[15:8]),  
  .acc_out  (mac_acc_out0[2*acc_width-1:acc_width])  
);
```

```
MAC u_MAC22 (  
  .clk      (clk),  
  .control  (control),  
  .data_in  (mac_data_in1[15:8]),  
  .wt_path_in (mac_wt_in1[15:8]),  
  .acc_in   (mac_acc_in1[2*acc_width-1:acc_width]),  
  .data_out  (mac_data_out1[15:8]),  
  .wt_path_out (mac_wt_out1[15:8]),  
  .acc_out  (mac_acc_out1[2*acc_width-1:acc_width])  
);
```



Per-cycle analysis - How the weights are loaded

t	wt_arr[31:24]	wt_arr[23:16]	wt_arr[15:8]	wt_arr[7:0]
0	A_{44}	A_{43}	A_{42}	A_{41}
1	A_{34}	A_{33}	A_{32}	A_{31}
2	A_{24}	A_{23}	A_{22}	A_{21}
3	A_{14}	A_{13}	A_{12}	A_{11}

$$A = \begin{bmatrix} A_{11} & A_{12} & A_{13} & A_{14} \\ A_{21} & A_{22} & A_{23} & A_{24} \\ A_{31} & A_{32} & A_{33} & A_{34} \\ A_{41} & A_{42} & A_{43} & A_{44} \end{bmatrix}$$

image source: https://semath.info/img/determinant_four_by_four_00.jpg

Per-cycle analysis - How the weights are loaded (example)

looks like:

```
initial begin
  @(posedge clk);
  control <= 1;
  wt_arr  <= 32'h05020304;

  @(posedge clk);
  wt_arr  <= 32'h03010203;

  @(posedge clk);
  wt_arr  <= 32'h07040102;

  @(posedge clk);
  wt_arr  <= 32'h01020403;
```

actual loaded matrix

$$\begin{pmatrix} 3 & 4 & 2 & 1 \\ 2 & 1 & 4 & 7 \\ 3 & 2 & 1 & 3 \\ 4 & 3 & 2 & 5 \end{pmatrix}$$

Per-cycle analysis - How the inputs are streamed in

t	data_arr[31:24]	data_arr[23:16]	data_arr[15:8]	data_arr[7:0]
4				A_{11}
5			A_{12}	A_{21}
6		A_{13}	A_{22}	A_{31}
7	A_{14}	A_{23}	A_{32}	A_{41}
8	A_{24}	A_{33}	A_{42}	
9	A_{34}	A_{43}		
10	A_{44}			
11				

$$A = \begin{bmatrix} A_{11} & A_{12} & A_{13} & A_{14} \\ A_{21} & A_{22} & A_{23} & A_{24} \\ A_{31} & A_{32} & A_{33} & A_{34} \\ A_{41} & A_{42} & A_{43} & A_{44} \end{bmatrix}$$

image source: https://semath.info/img/determinant_four_by_four_00.jpg

Per-cycle analysis - How the inputs are streamed (example)

looks like:

```
data_arr <= 32'h00 00 00 01;
data_arr <= 32'h00 00 01 02;
data_arr <= 32'h00 01 02 00;
data_arr <= 32'h00 01 01 00;
data_arr <= 32'h02 03 02 00;
data_arr <= 32'h04 01 00 00;
data_arr <= 32'h05 00 00 00;
```

actual loaded matrix

$$\begin{pmatrix} 1 & 1 & 1 & 0 \\ 2 & 2 & 1 & 2 \\ 0 & 1 & 3 & 4 \\ 0 & 2 & 1 & 5 \end{pmatrix}$$

*imagine a @(posedge clk) in
between each line

Per-cycle analysis - How the output is read as it streams out

t	acc_out[127:96]	acc_out[95:64]	acc_out[63:32]	acc_out[31:0]
8				A_{11}
9			A_{12}	A_{21}
10		A_{13}	A_{22}	A_{31}
11	A_{14}	A_{23}	A_{32}	A_{41}
12	A_{24}	A_{33}	A_{42}	
13	A_{34}	A_{43}		
14	A_{44}			
15				

$$A = \begin{bmatrix} A_{11} & A_{12} & A_{13} & A_{14} \\ A_{21} & A_{22} & A_{23} & A_{24} \\ A_{31} & A_{32} & A_{33} & A_{34} \\ A_{41} & A_{42} & A_{43} & A_{44} \end{bmatrix}$$

image source: https://semath.info/img/determinant_four_by_four_00.jpg

Per-cycle analysis - How the output is read as it streams out

looks like:

actual read matrix

```
[OUTPUT] time=4250 y3=0, y2=0, y1=0, y0=8
[OUTPUT] time=4750 y3=0, y2=0, y1=7, y0=21
[OUTPUT] time=5250 y3=0, y2=7, y1=18, y0=27
[OUTPUT] time=5750 y3=11, y2=17, y1=19, y0=27
[OUTPUT] time=6250 y3=29, y2=15, y1=19, y0=0
[OUTPUT] time=6750 y3=36, y2=19, y1=0, y0=0
[OUTPUT] time=7250 y3=42, y2=0, y1=0, y0=0
```

$$\begin{pmatrix} 8 & 7 & 7 & 11 \\ 21 & 18 & 17 & 29 \\ 27 & 19 & 15 & 36 \\ 27 & 19 & 19 & 42 \end{pmatrix}$$

*result tested & confirmed correct
with wolframalpha

Input:	Result:
$\begin{pmatrix} 1 & 1 & 1 & 0 \\ 2 & 2 & 1 & 2 \\ 0 & 1 & 3 & 4 \\ 0 & 2 & 1 & 5 \end{pmatrix} \cdot \begin{pmatrix} 3 & 4 & 2 & 1 \\ 2 & 1 & 4 & 7 \\ 3 & 2 & 1 & 3 \\ 4 & 3 & 2 & 5 \end{pmatrix}$	$\begin{pmatrix} 8 & 7 & 7 & 11 \\ 21 & 18 & 17 & 29 \\ 27 & 19 & 15 & 36 \\ 27 & 19 & 19 & 42 \end{pmatrix}$

Example of two input matrices streamed in one after another

```
data_arr <= 32'h00 00 00 01;
data_arr <= 32'h00 00 01 02;
data_arr <= 32'h00 01 02 00;
data_arr <= 32'h00 01 01 00;
data_arr <= 32'h02 03 02 01; // start of 2nd matrix
data_arr <= 32'h04 01 02 06;
data_arr <= 32'h05 03 07 01;
data_arr <= 32'h04 09 02 05;
data_arr <= 32'h0A 03 09 00;
data_arr <= 32'h04 0B 00 00;
data_arr <= 32'h01 00 00 00;
data_arr <= 32'h00 00 00 00;
```

```
[OUTPUT] time=4250 y3=0, y2=0, y1=0, y0=8
[OUTPUT] time=4750 y3=0, y2=0, y1=7, y0=21
[OUTPUT] time=5250 y3=0, y2=7, y1=18, y0=27
[OUTPUT] time=5750 y3=11, y2=17, y1=19, y0=27
[OUTPUT] time=6250 y3=29, y2=15, y1=19, y0=32
[OUTPUT] time=6750 y3=36, y2=19, y1=24, y0=99
[OUTPUT] time=7250 y3=42, y2=21, y1=79, y0=32
[OUTPUT] time=7750 y3=44, y2=69, y1=24, y0=70
[OUTPUT] time=8250 y3=132, y2=21, y1=54, y0=0
[OUTPUT] time=8750 y3=44, y2=59, y1=0, y0=0
[OUTPUT] time=9250 y3=106, y2=0, y1=0, y0=0
[OUTPUT] time=9750 y3=0, y2=0, y1=0, y0=0
```

*verification in wolframalpha

Input:

$$\begin{pmatrix} 1 & 1 & 1 & 0 \\ 2 & 2 & 1 & 2 \\ 0 & 1 & 3 & 4 \\ 0 & 2 & 1 & 5 \end{pmatrix} \cdot \begin{pmatrix} 3 & 4 & 2 & 1 \\ 2 & 1 & 4 & 7 \\ 3 & 2 & 1 & 3 \\ 4 & 3 & 2 & 5 \end{pmatrix}$$

Result:

$$\begin{pmatrix} 8 & 7 & 7 & 11 \\ 21 & 18 & 17 & 29 \\ 27 & 19 & 15 & 36 \\ 27 & 19 & 19 & 42 \end{pmatrix}$$

Input:

$$\begin{pmatrix} 1 & 2 & 3 & 4 \\ 6 & 7 & 9 & 10 \\ 1 & 2 & 3 & 4 \\ 5 & 9 & 11 & 1 \end{pmatrix} \cdot \begin{pmatrix} 3 & 4 & 2 & 1 \\ 2 & 1 & 4 & 7 \\ 3 & 2 & 1 & 3 \\ 4 & 3 & 2 & 5 \end{pmatrix}$$

Result:

$$\begin{pmatrix} 32 & 24 & 21 & 44 \\ 99 & 79 & 69 & 132 \\ 32 & 24 & 21 & 44 \\ 70 & 54 & 59 & 106 \end{pmatrix}$$

Some things to note

- This is just the matrix-multiply unit of the TPU. Many other components like:
 - The non-linear activation unit
 - Memory (on-chip and off-chip memory)
 - Control components that:
 - set up the system so you can load in weights for next layer while computing current layer
 - set up current layer's accumulations as the next layer's input
 - send the output back to the host
 - does the tiling
 - and more

The End

THANK YOU FOR LISTENING